



Master CHPS

Semestre 2, 2024-2025

Rapport de Debugging

TD n°1

Projet dbglib001

Etudiante :
Salma Bensmail

Professeur :
David Parrello

1 Introduction

Le debugging et le profiling sont des éléments essentiels du développement logiciel, en particulier pour les projets en langage C nécessitant une gestion optimisée des ressources. Ce TP se focalise sur l'analyse et la correction des erreurs dans un projet implémentant une bibliothèque numérique basée sur les BLAS (Basic Linear Algebra Subprograms). L'objectif est d'identifier et résoudre les erreurs de syntaxe avant d'exécuter et d'analyser le programme.

2 Installation et Compilation

2.1 Installation

Tout d'abord, nous avons téléchargé l'archive compressée contenant le projet et l'avons extraite à l'aide de la commande suivante :

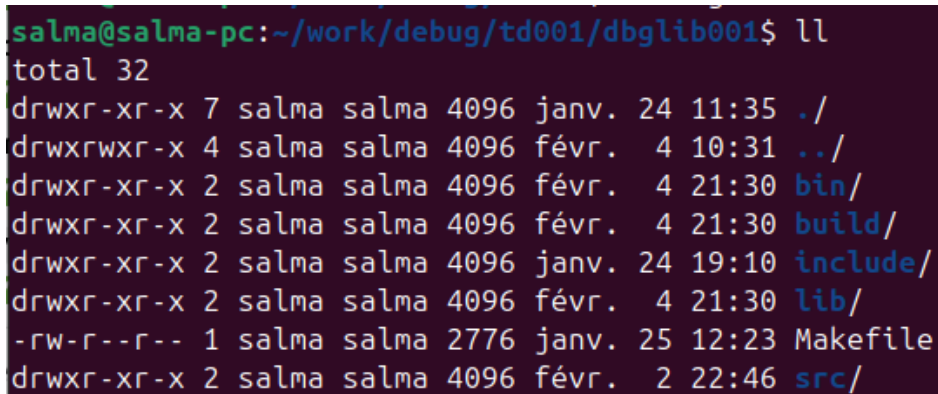
```
$ tar -xvzf dbg001.tar.gz
```

Cela a permis d'obtenir les fichiers sources, les fichiers d'en-tête et le Makefile, qui sert à automatiser la compilation.

Ensuite, nous avons vérifié la structure du projet en listant son contenu avec la commande suivante :

```
$ ll
```

Sortie de la commande :



```
salma@salma-pc:~/work/debug/td001/dbglib001$ ll
total 32
drwxr-xr-x 7 salma salma 4096 janv. 24 11:35 ./
drwxrwxr-x 4 salma salma 4096 févr.  4 10:31 ../
drwxr-xr-x 2 salma salma 4096 févr.  4 21:30 bin/
drwxr-xr-x 2 salma salma 4096 févr.  4 21:30 build/
drwxr-xr-x 2 salma salma 4096 janv. 24 19:10 include/
drwxr-xr-x 2 salma salma 4096 févr.  4 21:30 lib/
-rw-r--r-- 1 salma salma 2776 janv. 25 12:23 Makefile
drwxr-xr-x 2 salma salma 4096 févr.  2 22:46 src/
```

FIGURE 1 – Structure du projet obtenue avec la commande ll

Nous constatons que le projet est bien organisé avec les dossiers nécessaires :

- **src/** : Contient les fichiers source du projet
- **include/** : Regroupe les fichiers d'en-tête définissant les prototypes des fonctions
- **bin/** : Stockera l'exécutable final
- **build/** : Contendra les fichiers objets intermédiaires
- **lib/** : Regroupe les bibliothèques compilées
- **Makefile** : Fichier permettant d'automatiser la compilation du projet

2.2 Compilation et Exécution

Avant de compiler, nous avons exécuté la commande `make clean` afin de supprimer les fichiers objets et les exécutables précédemment générés. Cela permet d'assurer une compilation propre.

```
$ make clean
```

Sortie de la commande :

```
salma@salma-pc:~/work/debug/td001/dbglib001$ make clean
rm -f ./build/*.o ./lib/*.a ./bin/*.exe src/*~ include/*~
```

Ensuite, nous avons lancé la compilation avec la commande `make` :

```
$ make
```

Sortie de la commande :

```
salma@salma-pc:~/work/debug/td001/dbglib001$ make
gcc -I./include -std=gnu99 -g src/cblasl0.c -c -o build/cblasl0.o
gcc -I./include -std=gnu99 -g src/cblasl1.c -c -o build/cblasl1.o
ar -rc lib/dbglib.a build/cblasl0.o build/cblasl1.o
gcc -I./include -std=gnu99 -g src/test-001.c -c -o build/test-001.o
gcc -I./include -std=gnu99 -g build/test-001.o lib/dbglib.a -o bin/test-001.exe
```

La compilation s'est terminée avec succès, et l'exécutable `test-001.exe` a été généré sans erreur majeure.

Nous avons ensuite utilisé `gdb` pour exécuter et analyser notre programme :

```
$ gdb bin/test-001.exe
```

Ensuite, nous avons exécuté le programme à l'aide de la commande `run` dans `gdb` :

```
(gdb) run
```

Sortie de la commande :

```
(gdb) run
Starting program: /home/salma/work/debug/td001/dbglib001/bin/test-001.exe

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Entrer la taille des vecteurs (exemple 8): 1800

Program received signal SIGSEGV, Segmentation fault.
Download failed: Argument invalide. Continuing without source file ./stdio-common/vfscanf-internal.c.
0x00007ffff7c70136 in __vfscanf_internal (s=<optimized out>, format=<optimized out>, argptr=argptr@entry=0x7fffffffdb80,
mode_flags=mode_flags@entry=2) at ./stdio-common/vfscanf-internal.c:1976
warning: 1976 ./stdio-common/vfscanf-internal.c: Aucun fichier ou dossier de ce nom
```

Le programme `test-001.exe` a été exécuté dans GDB (GNU Debugger) et a généré une erreur de segmentation (**Segmentation Fault, SIGSEGV**).

Afin de mieux comprendre la source de l'erreur, nous avons exécuté la commande `bt` (backtrace) pour afficher la pile d'exécution au moment du crash.

Sortie de la commande :

```
(gdb) bt
#0 0x00007ffff7c70136 in __vfscanf_internal (s=<optimized out>, format=<optimized out>, argptr=argptr@entry=0x7ffffffdb80, mode_flags=mode_flags@entry=2) at ./stdio-common/vfscanf-internal.c:1976
#1 0x00007ffff7c5feb6 in __isoc99_scanf (format=<optimized out>) at ./stdio-common/isoc99_scanf.c:30
#2 0x000055555555233 in main (argc=1, argv=0x7ffffffdc8) at src/test-001.c:11
```

2.3 Analyse du Backtrace

Frame #0 (__vfscanf_internal)

```
#0 0x00007ffff7c70136 in __vfscanf_internal (s=<optimized out>, format=<
↳ optimized out>,
  argptr=argptr@entry=0x7ffffffdb80, mode_flags=mode_flags@entry=2)
  at ./stdio-common/vfscanf-internal.c:1976
```

L'erreur s'est produite dans `__vfscanf_internal`, une fonction interne utilisée par `scanf` pour lire l'entrée utilisateur. Cela indique que le programme a crashé pendant un appel à `scanf`.

Frame #1 (__isoc99_scanf)

```
#1 0x00007ffff7c5feb6 in __isoc99_scanf (format=<optimized out>)
  at ./stdio-common/isoc99_scanf.c:30
```

`__isoc99_scanf` est la version standard de `scanf`. Cette ligne confirme que le problème vient bien de `scanf`, qui a tenté d'écrire dans une adresse mémoire invalide.

Frame #2 (main dans test-001.c, ligne 11)

```
#2 0x000055555555233 in main (argc=1, argv=0x7ffffffdc8) at src/test-001.c:11
```

Le crash s'est produit dans la fonction `main` du fichier `test-001.c`, ligne 11. Cela signifie que la ligne contenant `scanf` dans `test-001.c` (ligne 11) est responsable du crash.

3 Debugging : les erreurs de syntaxe

Bug # - Segmentation Fault dans test-001.c

- **Localisation** : `test-001.c`, ligne 11
- **Importance** : Critique (Crash du programme)
- **Problème** : La variable `pSize` est initialisée à `NULL` et utilisée dans `scanf`, ce qui provoque un accès mémoire invalide

Code source avec erreur :

```
7 int Size;
8 int *pSize=0; // ERREUR : pSize est NULL, cela cause un Segmentation Fault
9
10 printf("Entrer la taille des vecteurs (exemple 8): ");
11 scanf("%d", pSize); // ERREUR : pSize est NULL, cause un Segmentation Fault
```

Correction apportée :

```
7   int Size;
8   int *pSize= &Size;// Maintenant, pSize pointe vers une variable valide
9
10  printf("Entrer la taille des vecteurs (exemple 8): ");
11  scanf("%d", pSize);// scanf écrit dans Size au lieu d'un pointeur NULL
```

- **Explication :**

- La version initiale tentait d'écrire directement dans un pointeur non initialisé
- Allocation d'une variable stack `Size` pour stocker le résultat
- Le pointeur `pSize` devient maintenant une référence valide vers cette variable
- Résolution du SIGSEGV car l'accès mémoire est maintenant légitime

4 Debugging : les erreurs de segmentation critiques à l'exécution

Bug # - Segmentation Fault dans `cblas1.c`

Après avoir corrigé l'erreur initiale située à la ligne 11 de `test-001.c`, nous avons recompilé et exécuté à nouveau le programme. Cependant, une nouvelle erreur de segmentation (SIGSEGV) a été détectée lors de l'exécution :

```
(gdb) run
```

Sortie de la commande :

```
(gdb) run
Starting program: /home/salma/work/debug/td001/dbglib001/bin/test-001.exe

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Entrer la taille des vecteurs (exemple 8): 8

Program received signal SIGSEGV, Segmentation fault.
scopy (size=0, x=0x0, incx=1, y=0x7fffffffdd80, incy=1) at src/cblas1.c:41
41      y[i] = x[i];
```

- **Localisation :** `cblas1.c`, ligne 41
- **Importance :** Critique (Crash du programme)
- **Problème :**

Le problème réside dans le code :

```
float *sv3;
```

du fichier `test-001.c`.

- `sv3` est un pointeur non initialisé.
- L'appel à `scopy(Size, sv2, 1, sv3, 1)`; tente d'écrire dans `sv3`, qui pointe vers une adresse non définie, ce qui cause un **Segmentation Fault**.

Correction Recommandée

Avant d'utiliser `sv3`, il faut l'allouer correctement :

```
float *sv3 = malloc(Size);
```

```

10  printf("Entrer la taille des vecteurs (exemple 8): ");
11  scanf("%d", pSize);
12
13  // Création des vecteurs
14  float *sv1=salloc(Size);
15  float *sv2=salloc(Size);
16  float *sv3=salloc(Size); //float *sv3;

```

Cela garantit que `sv3` pointe vers une mémoire valide avant son utilisation dans `scopy`.

Exécution réussie après correction :

Après avoir corrigé l'erreur et recompilé le programme, nous avons exécuté celui-ci en entrant une taille de vecteur de 4. Voici le résultat obtenu :

```

(gdb) run
Starting program: /home/salma/work/debug/td001/dbglib001/bin/test-001.exe
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Entrer la taille des vecteurs (exemple 8): 4

=====
Vectors before saxpy computations :
[ 1.000, 2.000, 3.000, 4.000 ]
[ 1.000, 2.000, 3.000, 4.000 ]
[ 1.000, 2.000, 3.000, 4.000 ]

=====
Vectors after saxpy computations :
[ 1.000, 2.000, 3.000, 4.000 ]
[ 1.000, 2.000, 3.000, 4.000 ]
[ 2.000, 4.000, 6.000, 8.000 ]

=====
Vectors after scopy computations :
[ 2.000, 4.000, 6.000, 8.000 ]
[ 1.000, 2.000, 3.000, 4.000 ]
[ 2.000, 4.000, 6.000, 8.000 ]

```

- Exécution sans erreur avec sortie valide
- Vecteurs correctement initialisés et manipulés
- Opérations BLAS (`saxpy` et `scopy`) fonctionnelles

5 Debugging : les erreurs à l'exécution cachées

Nous utiliseront l'outil d'analyse debug `valgrind` pour identifier les erreurs.

5.1 Installation de Valgrind

L'outil **Valgrind** est utilisé pour détecter les erreurs de mémoire dans les programmes. Il peut être installé avec la commande suivante :

```
sudo apt update && sudo apt install valgrind
```

On peut vérifier la version avec :

```
valgrind --version
```

```
salma@salma-pc:~/work/debug/td001/dbglib001$ valgrind --version
valgrind-3.22.0
```

5.2 Exécution du programme avec Valgrind

Pour exécuter `test-001.exe` sous Valgrind et vérifier les erreurs de mémoire, la commande utilisée est :

```
valgrind --leak-check=full --track-origins=yes ./bin/test-001.exe
```

- `-leak-check=full` : Vérifie les fuites de mémoire.
- `-track-origins=yes` : Identifie l'origine des erreurs mémoire.
- `./bin/test-001.exe` : Exécute le programme sous Valgrind.

Sortie de commande :

```
salma@salma-pc:~/work/debug/td001/dbglib001$ valgrind --leak-check=full --track-origins=yes ./bin/test-001.exe
==5614== Memcheck, a memory error detector
==5614== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==5614== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==5614== Command: ./bin/test-001.exe
==5614==
Entrer la taille des vecteurs (exemple 8): 8
==5614== Invalid read of size 4
==5614== at 0x10991D: scopy (cblas1.c:41)
==5614== by 0x1092D1: main (test-001.c:22)
==5614== Address 0x4a83a20 is 0 bytes after a block of size 32 alloc'd
==5614== at 0x484E366: posix_memalign (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==5614== by 0x109476: salloc (cblas0.c:16)
==5614== by 0x109259: main (test-001.c:15)
==5614==
==5614== Invalid write of size 4
==5614== at 0x109921: scopy (cblas1.c:41)
==5614== by 0x1092D1: main (test-001.c:22)
==5614== Address 0x4a83a00 is 0 bytes after a block of size 32 alloc'd
==5614== at 0x484E366: posix_memalign (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==5614== by 0x109476: salloc (cblas0.c:16)
==5614== by 0x109267: main (test-001.c:16)
==5614==

=====
Vectors before saxpy computations :
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]
==5614== Invalid read of size 4
==5614== at 0x10990E: saxpy (cblas1.c:64)
==5614== by 0x10934C: main (test-001.c:30)
==5614== Address 0x4a83ae0 is 0 bytes after a block of size 32 alloc'd
==5614== at 0x484E366: posix_memalign (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==5614== by 0x109476: salloc (cblas0.c:16)
==5614== by 0x109267: main (test-001.c:16)
==5614==
==5614== Invalid read of size 4
==5614== at 0x1099F6: saxpy (cblas1.c:64)
==5614== by 0x10934C: main (test-001.c:30)
==5614== Address 0x4a83920 is 0 bytes after a block of size 32 alloc'd
==5614== at 0x484E366: posix_memalign (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==5614== by 0x109476: salloc (cblas0.c:16)
==5614== by 0x10924B: main (test-001.c:14)
==5614==
==5614== Invalid write of size 4
==5614== at 0x109A17: saxpy (cblas1.c:64)
==5614== by 0x10934C: main (test-001.c:30)
==5614== Address 0x4a83ae0 is 0 bytes after a block of size 32 alloc'd
==5614== at 0x484E366: posix_memalign (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==5614== by 0x109476: salloc (cblas0.c:16)
==5614== by 0x109267: main (test-001.c:16)
==5614==
```

```

=====
Vectors after saxpy computations :
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]
[ 2.000, 4.000, 6.000, 8.000, 10.000, 12.000, 14.000, 16.000 ]
==5614== Invalid read of size 4
==5614==    at 0x10991D: scopy (cblas1.c:41)
==5614==    by 0x10938D: main (test-001.c:38)
==5614== Address 0x4a83ae0 is 0 bytes after a block of size 32 alloc'd
==5614==    at 0x484E366: posix_memalign (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==5614==    by 0x109476: salloc (cblas10.c:16)
==5614==    by 0x109267: main (test-001.c:16)
==5614==
==5614== Invalid write of size 4
==5614==    at 0x109921: scopy (cblas1.c:41)
==5614==    by 0x10938D: main (test-001.c:38)
==5614== Address 0x4a83920 is 0 bytes after a block of size 32 alloc'd
==5614==    at 0x484E366: posix_memalign (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==5614==    by 0x109476: salloc (cblas10.c:16)
==5614==    by 0x10924B: main (test-001.c:14)
==5614==

```

5.3 Correction dans test-001.c

```

4  int main(int argc, char **argv) {
5      int Size;
6      int *pSize = &Size;
7
8      printf("Entrer la taille des vecteurs (exemple 8): ");
9      scanf("%d", pSize);
10
11     // Vérification de la taille
12     if (Size <= 0) {
13         printf("Erreur : La taille doit être > 0.\n");
14         return 1;
15     }
16
17     // Création des vecteurs avec allocation mémoire
18     float *sv1 = salloc(Size);
19     float *sv2 = salloc(Size);
20     float *sv3 = salloc(Size);
21
22     // Vérification de l'allocation mémoire
23     if (sv1 == NULL || sv2 == NULL || sv3 == NULL) {
24         printf("Erreur : allocation mémoire échouée !\n");
25         return 1;
26     }
27
28     // Initialisation des Vecteurs
29     sinit(Size, sv1, 1.0, 1.0);
30     sinit(Size, sv2, 1.0, 1.0);
31     scopy(Size, sv2, 1, sv3, 1);

```



```

33     printf("\n\n=====\\n");
34     printf("Vectors before saxpy computations : \\n");
35     sdump(Size, sv1);
36     sdump(Size, sv2);
37     sdump(Size, sv3);
38
39     // Vérification avant d'utiliser saxpy
40     if (sv1 != NULL && sv3 != NULL) {
41         | saxpy(Size, 1.0, sv1, 1, sv3, 1);
42     }
43
44     printf("\n\n=====\\n");
45     printf("Vectors after saxpy computations : \\n");
46     sdump(Size, sv1);
47     sdump(Size, sv2);
48     sdump(Size, sv3);
49
50     // Libération mémoire pour éviter les fuites
51     free(sv1);
52     free(sv2);
53     free(sv3);
54
55     return 0;
56 }

```

Explication des corrections

Vérification de Size

- Ajout d'une vérification `if (Size <= 0)` pour éviter des allocations inutiles.

Ajout de vérifications avant `scopy` et `saxpy`

- On s'assure que `sv1`, `sv2`, et `sv3` ne sont pas `NULL` avant de les utiliser.

Libération mémoire (`sfree()`)

- Ajout de `sfree(sv1)`, `sfree(sv2)`, `sfree(sv3)` pour éviter les **fuites mémoire**.

5.4 Erreurs détectées dans `cblasl1.c`

L'outil Valgrind a révélé deux erreurs critiques de gestion mémoire :

- **Fonction `scopy()`** - Ligne 41
 - Type : Invalid read of size 4
 - Cause : Accès mémoire hors limites
 - Code fautif : `for (int i = 0; i <= size; ++i)`

```

==5614== Invalid read of size 4
==5614==    at 0x10991D: scopy (cblasl1.c:41)
==5614==    by 0x1092D1: main (test-001.c:22)

```

- **Fonction `saxpy()`** - Ligne 64
 - Type : Invalid read of size 4
 - Cause : Indexation dépassant la mémoire allouée
 - Code fautif : `for (int i = 0; i <= size; ++i)`

```

==5614== Invalid read of size 4
==5614==    at 0x1099DE: saxpy (cblasl1.c:64)
==5614==    by 0x10934C: main (test-001.c:30)

```

Corrections implémentées

Correction dans Scopy(),(fichier cblasl1.c) :

```

33  }
34
35  void scopy(const int size, const float *x, const int incx, float *y, const int incy)
36  {
37      if ( (incx == 1) && (incy == 1) )
38      {
39          for (int i=0; i< size; ++i) // i < size au lieu de i <= size
40          {

```

Correction dans Saxpy(),(fichier cblasl1.c) :

```

59  if ( (incx == 1) && (incy == 1) )
60  {
61      for (int i=0; i< size; ++i) // i < size au lieu de i <= size
62      {
63          y[i] = y[i] + alpha * x[i]; // ✅ Accès sécurisé dans les limites du tableau
64      }
65  }
66

```

Vérification finale après corrections

Après les corrections dans `cblasl1.c` et `test-001.c`, nous avons recompilé et exécuté le programme sous Valgrind pour s'assurer qu'il ne subsistait plus d'erreurs mémoire.

Résultats de l'exécution :

Affichage des Vecteurs Avant et Après saxpy()

- **Avant saxpy()** : Les valeurs des vecteurs sont correctement initialisées.
- **Après saxpy()** : Les calculs vectoriels sont effectués correctement.

Rapport Mémoire par Valgrind

- **Heap summary :**

```
in use at exit: 0 bytes in 0 blocks
```

→ Indique que toute la mémoire allouée a été libérée.

- **Total heap usage :**

```
total heap usage: 5 allocs, 5 frees, 2,144 bytes allocated
```

→ Vérifie que l'allocation et la libération de mémoire sont bien équilibrées.

- **Absence de fuites mémoire :**

```
==8676== All heap blocks were freed -- no leaks are possible
```

→ Confirme que toutes les allocations ont été libérées correctement.

- **Aucune erreur détectée :**

```
==8676== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

→ Indique qu'il n'y a plus d'erreurs mémoire après correction.

```

salna@salna-pc:~/work/debug/td001/dbglib001$ make clean
rm -f ./build/*.o ./lib/*.a ./bin/*.exe src/*~ include/*~
salna@salna-pc:~/work/debug/td001/dbglib001$ make
gcc -I./include -std=gnu99 -g src/cblasl0.c -c -o build/cblasl0.o
gcc -I./include -std=gnu99 -g src/cblasl1.c -c -o build/cblasl1.o
ar -rc lib/dbglib.a build/cblasl0.o build/cblasl1.o
gcc -I./include -std=gnu99 -g src/test-001.c -c -o build/test-001.o
gcc -I./include -std=gnu99 -g build/test-001.o lib/dbglib.a -o bin/test-001.exe
salna@salna-pc:~/work/debug/td001/dbglib001$ valgrind --leak-check=full --track-origins=yes ./bin/test-001.exe
==8676== Memcheck, a memory error detector
==8676== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==8676== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==8676== Command: ./bin/test-001.exe
==8676==
Entrer la taille des vecteurs (exemple 8): 8

```

```

=====
Vectors before saxpy computations :
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]

=====
Vectors after saxpy computations :
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]
[ 1.000, 2.000, 3.000, 4.000, 5.000, 6.000, 7.000, 8.000 ]
[ 2.000, 4.000, 6.000, 8.000, 10.000, 12.000, 14.000, 16.000 ]
==8676==
==8676== HEAP SUMMARY:
==8676==    in use at exit: 0 bytes in 0 blocks
==8676==   total heap usage: 5 allocs, 5 frees, 2,144 bytes allocated
==8676==
==8676== All heap blocks were freed -- no leaks are possible
==8676==
==8676== For lists of detected and suppressed errors, rerun with: -s
==8676== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

6 Debugging : les erreurs dissimulées

Dans cet exercice, nous avons pour objectif d'effectuer une analyse approfondie du code afin d'identifier des erreurs non détectées par les outils de debugging classiques comme GDB et Valgrind.

Ce travail reste essentiel pour garantir la fiabilité du programme en repérant des erreurs **logiques** ou **structurelles** qui pourraient ne pas provoquer d'erreurs immédiates mais qui pourraient impacter le comportement du programme.

Une future analyse pourrait inclure :

- Une **revue manuelle du code** pour identifier d'éventuelles incohérences ou erreurs logiques.
- La mise en place de **tests unitaires** afin de valider le bon fonctionnement des différentes fonctions.

7 Conclusion

Ce travail a permis d'analyser et de corriger plusieurs erreurs critiques liées à la gestion de la mémoire et aux accès hors limites des tableaux. Grâce à l'utilisation de GDB et Valgrind, nous avons pu identifier les causes des erreurs de segmentation et des fuites mémoire, et y apporter des corrections adaptées. La compilation et l'exécution du programme sont désormais stables, et aucune fuite mémoire n'a été détectée. Cela met en évidence l'importance des outils de debugging pour garantir la robustesse et la fiabilité du code en environnement C.